COS120 - CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

BÀI 5: NGĂN XẾP VÀ HÀNG ĐỢI

MỤC TIÊU

- Hiểu và phân biệt giữa ngăn xếp (**stack**) và hàng đợi (**queue**).
- Cài đặt được ngăn xếp và hàng đợi bằng mảng và danh sách **liên kết.**
- Thực hiện các thao tác cơ bản của ngăn xếp và hàng đợi.
- Hiểu được tính ứng dụng thực tế của ngăn xếp và hàng đợi.
- Có khả năng áp dụng ngăn xếp và hàng đợi trong các **bài toán lập trình**.

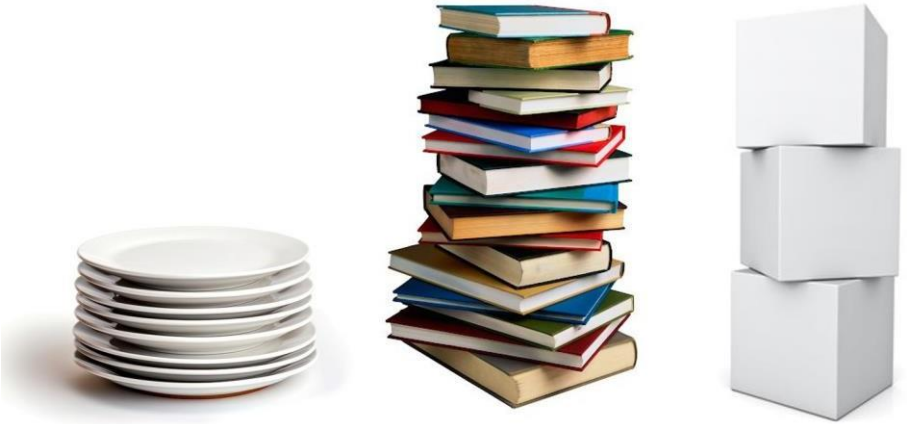
NỘI DUNG

- 5.1 - NGĂN XẾP - **STACK**
- 5.2 - HÀNG ĐỢI - **QUEUE**

NGĂN XẾP VÀ HÀNG ĐỢI

5.1 - NGĂN XẾP (STACK)

KHÁI NIỆM VỀ NGĂN XẾP (STACK)

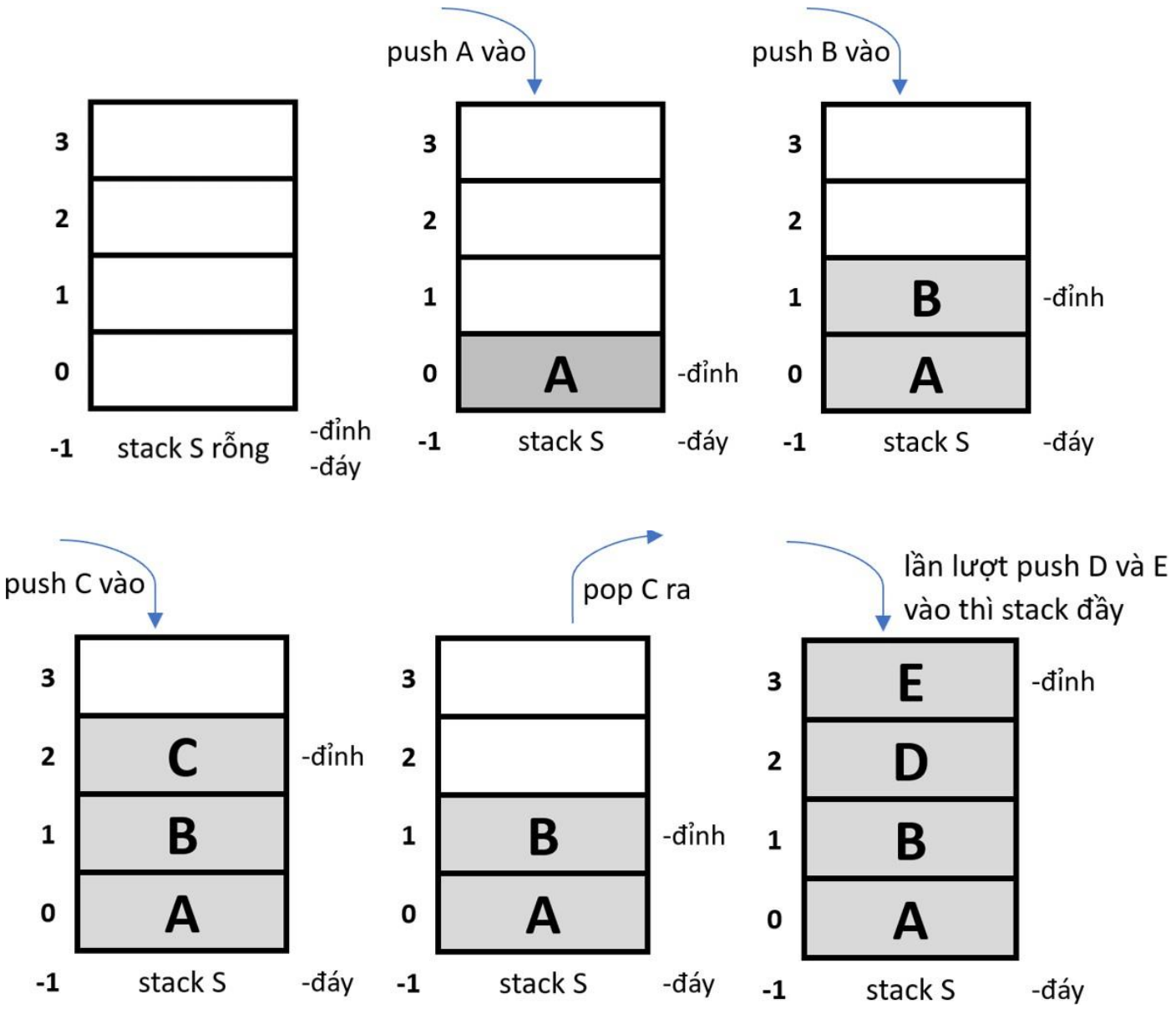


- Khi ta xếp vào một chồng, thì phần tử này sẽ đặt lên trên phần tử kia.
- Phần tử vào trước bị phần tử vào sau đặt chồng lên.
- Khi lấy ra thì phải lấy phần tử trên đỉnh rồi mới lấy từng phần tử tiếp theo.
- Cơ chế như thế được gọi là **Last In First Out** (LIFO)
- Stack (ngăn xếp) có thể được xem là một dạng danh sách đặc biệt.
- Tác vụ **thêm** vào hoặc **lấy** ra một phần tử chỉ diễn ra ở **đỉnh** stack.

KHÁI NIỆM NGĂN XẾP

MINH HỌA CƠ CHẾ LIFO CỦA STACK

- Stack có kích thước 4 phần tử



HAI PHƯƠNG PHÁP HIỆN THỰC NGĂN XẾP

❑ SỬ DỤNG DANH SÁCH LIỀN KỀ (Array-Based Stack)

- Mô tả: stack được biểu diễn dưới dạng một **mảng cố định**.
- Một biến chỉ số (**top**) để theo dõi đỉnh của stack.
- Ưu điểm: truy cập phần tử nhanh chóng (**O(1)**). Dễ cài đặt.
- Nhược điểm: kích thước **cố định**, **khó** thay đổi kích thước.

❑ SỬ DỤNG DANH SÁCH LIÊN KẾT (Linked List-Based Stack)

- Mô tả: stack là một danh sách liên kết đơn.
- **Mỗi** phần tử là một nút trong DS.
- Ưu điểm: kích thước động, thay đổi linh động số lượng phần tử.
- Nhược điểm: truy cập **chậm** do duyệt các phần tử qua con trỏ.

HAI PHƯƠNG PHÁP HIỆN THỰC NGĂN XẾP

CÁC TÁC VỤ CƠ BẢN TRÊN STACK

- Khởi tạo stack (Initialize)
- Kiểm tra stack rỗng (IsEmpty)
- Kiểm tra stack đầy (IsFull) - chỉ áp dụng cho stack dựa trên mảng
- Thêm phần tử vào stack (Push)
- Loại bỏ phần tử khỏi stack (Pop)
- Truy cập phần tử trên cùng của stack (Peek)

HIỆN THỰC BẰNG DANH SÁCH LIỀN KỀ

Hiện thực stack bằng mảng một chiều (**danh sách liền kề**) có thể thực hiện theo hai cách:

- **Kích thước cố định (fixed size)**
  - Ưu điểm: Không cần quản lý bộ nhớ động.
  - Nhược điểm: Kích thước giới hạn, có thể gây lãng phí bộ nhớ.
- **Kích thước thay đổi (dynamic size)**
  - Ưu điểm: linh hoạt về kích thước, tối ưu sử dụng bộ nhớ.
  - Nhược điểm: phức tạp hơn trong cài đặt và quản lý.

XÂY DỰNG CẤU TRÚC DỮ LIỆU

- Cấu trúc một stack gồm 3 thành phần:
  - **Mảng một chiều arr** chứa các phần tử.
  - Biến **top** để quản lý **đỉnh** của stack.
  - Biến **size** quản lý kích thước của stack.

```
// Định nghĩa cấu trúc Stack
struct Stack{
    int *arr;   // Mảng dùng để chứa các phần tử của stack
    int top;    // Chỉ số của phần tử tại đỉnh stack
    int size;   // Kích thước (khả năng lưu trữ) của stack
};
typedef struct Stack STACK; //đặt là STACK cho dễ gọi
```

XÂY DỰNG CÁC TÁC VỤ

KHỞI TẠO NGĂN XẾP

- Cấp phát bộ nhớ cho mảng động và khởi tạo các biến top và size.

```
void Initialize(STACK &stack, int capacity) {
    stack.arr = (int*)malloc(capacity * sizeof(int));
    if (stack.arr == NULL)
    {
        printf("Memory allocation failed.\n");
        exit(0);
    }
    stack.top = -1;
    stack.size = capacity;
}
```

XÂY DỰNG CÁC TÁC VỤ

KIỂM TRA NGĂN XẾP RỖNG

- Khi kiểm tra ngăn xếp (stack) rỗng chỉ cần so sánh biến top với -1.

```
int IsEmpty(STACK stack)
{
    return stack.top == -1;
}
```

KIỂM TRA NGĂN XẾP ĐẦY

- Kiểm tra biến top có chỉ đến phần tử cuối cùng chưa (chỉ số là size-1).

```
int IsFull(STACK stack)
{
    return stack.top == stack.size-1;
}
```

XÂY DỰNG CÁC TÁC VỤ

THÊM PHẦN TỬ VÀO NGĂN XẾP - TÁC VỤ PUSH

- Tác vụ thêm còn được gọi là tác vụ push.
- Nếu stack bị đầy thì ta không thể thêm.

```
void Push(STACK &stack, int value)
{
    if (IsFull(stack))
    {
        printf("Stack is full.\n");
        return;
    }
    stack.arr[++stack.top] = value;
}
```

XÂY DỰNG CÁC TÁC VỤ

THÊM PHẦN TỬ VÀO NGĂN XẾP KHI NÓ BỊ ĐẦY

- Khi stack đầy mà ta vẫn muốn thêm phần tử.
- Thì ta có thể tiến hành cấp phát thêm vùng nhớ cho mảng arr của stack.

```
void Push(STACK &stack, int value) {
    if (IsFull(stack)) {
        // Tăng gấp đôi kích thước của stack khi nó bị đầy
        stack.size *= 2;
        stack.arr=(int*)realloc(stack.arr,stack.size*sizeof(int));
        if (stack.arr == NULL) {
            printf("Memory reallocation failed\n");
            return;
        }
    }
    stack.arr[++stack.top] = value;
}
```

XÂY DỰNG CÁC TÁC VỤ

LẤY MỘT PHẦN TỬ RA KHỎI NGĂN XẾP - TÁC VỤ POP

- Tác vụ pop sẽ lấy một phần tử tại đỉnh (top) khỏi ngăn xếp.
- Nếu stack rỗng thì tác vụ pop không thành công.
- Giá trị lấy ra chứa trong value.
- Pop xong, thì top phải giảm chỉ số xuống.

```
void Pop(STACK &stack, int &value) {
    if (IsEmpty(stack)) {
        printf("Stack underflow.\n");
        return;
    }
    value = stack.arr[stack.top--];
}
```

XÂY DỰNG CÁC TÁC VỤ

TRUY CẬP PHẦN TỬ TRÊN CÙNG – TÁC VỤ PEEK

- Gọi là tác vụ peek, sẽ trả về giá trị phần tử tại top của stack, nó **không** làm giảm biến top.

```
void Peek(STACK stack, int &value) {
    if (IsEmpty(stack)) {
        printf("Stack is empty.\n"); return;
    }
    value = stack.arr[stack.top];
}
```

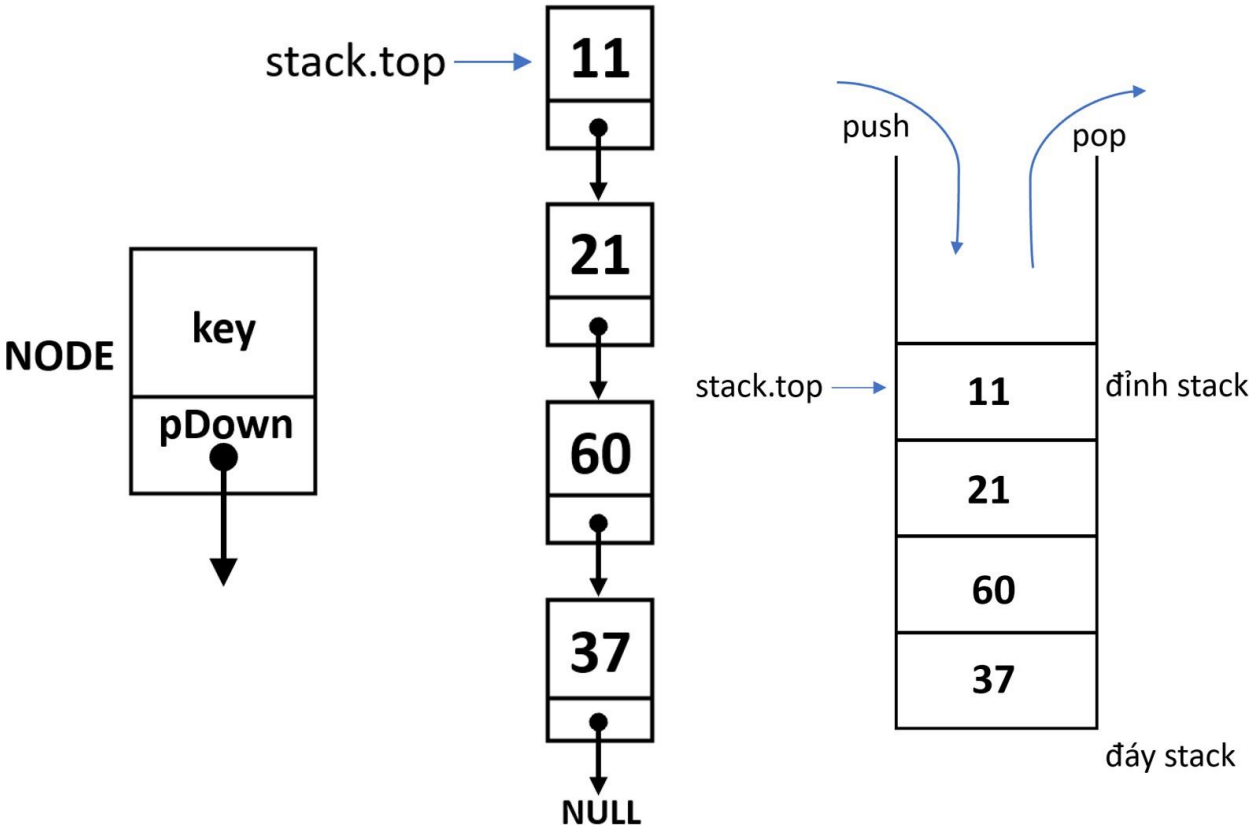
GIẢI PHÓNG BỘ NHỚ CỦA NGĂN XẾP

- Nếu không còn dùng stack thì phải giải phóng bộ nhớ đã cấp cho stack.

```
void FreeStack(STACK &stack) {
    free(stack.arr);
}
```

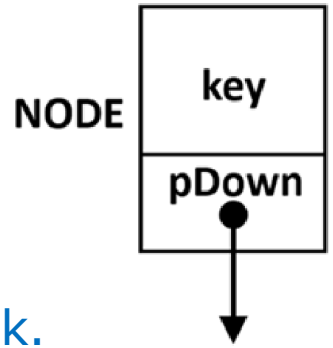
HIỆN THỰC BẰNG DANH SÁCH LIÊN KẾT ĐƠN

- Con trỏ **top** quản lý đỉnh của stack (tương đương nút head trong DSLK đơn).
- Tác vụ **pop** tương đương với xóa nút head trong DSLK đơn.
- Đáy stack chính là giá trị NULL của con trỏ tại nút cuối của DS.



XÂY DỰNG CẤU TRÚC DỮ LIỆU

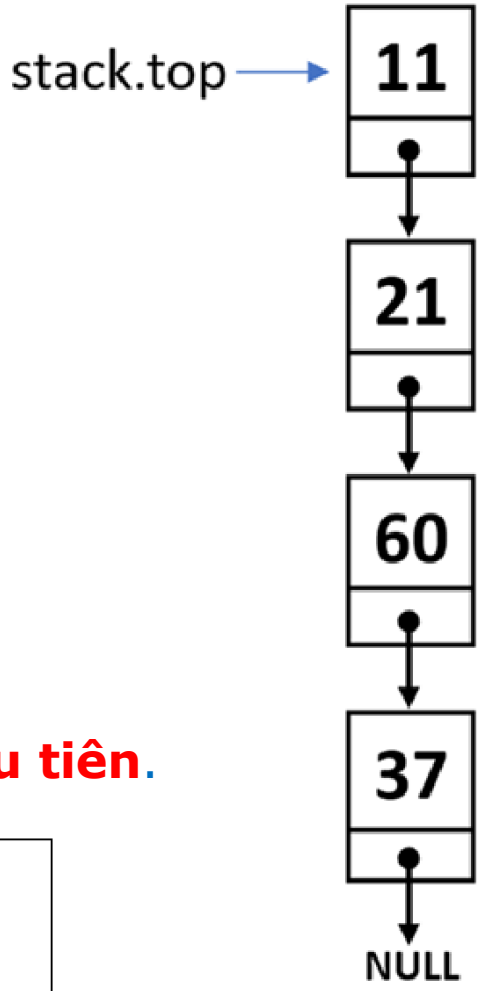
- Một phần tử của stack gồm hai thành phần:
  - Dữ liệu (key): chứa giá trị của phần tử.
  - Con trỏ (pDown): trỏ tới nút bên dưới trong stack.




```
typedef struct Node {
    int key;
    struct Node* pDown;
} NODE;
```

- Cấu trúc dữ liệu của stack sẽ chỉ cần một con trỏ **top** để quản lý nút **đầu tiên**.

```
typedef struct {
    NODE* top;
} STACK;
```



XÂY DỰNG CÁC TÁC VỤ

KHỞI TẠO NGĂN XẾP

- Đặt **top** của stack là NULL, biểu thị rằng stack rỗng.

```
void Initialize(STACK &stack) {
    stack.top = NULL;
}
```

KIỂM TRA NGĂN XẾP RỖNG

- Ta chỉ cần kiểm tra con trỏ top của nó có bằng NULL hay không.

```
int IsEmpty(STACK stack) {
    return stack.top == NULL;
}
```

XÂY DỰNG CÁC TÁC VỤ

KIỂM TRA NGĂN XẾP ĐẦY

- Bộ nhớ của một máy tính rất khó mà hết khi cấp phát một nút mới cho ngăn xếp (stack).
- Cho nên hiện thực stack bằng danh sách liên kết thì tác vụ kiểm tra stack đầy là **không cần thiết**.
- Ngày xưa, dung lượng RAM của máy tính rất ít và dễ hết, cho nên nếu muốn viết, thì ta có thể làm như sau

```
int IsFull(STACK stack) {
    NODE* newNode = (NODE*)malloc(sizeof(NODE));
    if (newNode == NULL) {
        // Không thể cấp phát bộ nhớ cho node mới
        // Stack được coi là đầy
        return 1; // Trả về true
    }
    else {
        free(newNode); // Giải phóng bộ nhớ của node mới
        return 0; // Trả về false
    }
}
```

XÂY DỰNG CÁC TÁC VỤ

THÊM PHẦN TỬ VÀO NGĂN XẾP

- Cấp phát bộ nhớ cho nút mới.
- Đặt key và pDown cho nút mới.
- Gắn nút mới vào đỉnh của stack
- Cập nhật lại con trỏ top trỏ đến nút mới.

```
void Push(STACK &stack, int value) {
    NODE* newNode = (NODE*)malloc(sizeof(NODE));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        return;
    }
    newNode->key = value;
    newNode->pDown = stack.top;
    stack.top = newNode;
}
```

XÂY DỰNG CÁC TÁC VỤ

LẤY MỘT PHẦN TỬ RA KHỎI NGĂN XẾP

- Lưu lại **key** của node đỉnh vào **tham chiếu value**.
- **Cập nhật top** tới **node kế tiếp**.
- Giải phóng bộ nhớ của node đỉnh.

```
void Pop(STACK &stack, int &value) {
    if (IsEmpty(stack)) {
        printf("Stack underflow\n");
        return ;
    }
    NODE* temp = stack.top;
    value = temp->key;
    stack.top = stack.top->pDown;
    free(temp);
}
```

XÂY DỰNG CÁC TÁC VỤ

TRUY CẬP PHẦN TỬ TRÊN CÙNG

- Tác vụ peek: lưu key của top vào value và không thay đổi top.

```
void Peek(STACK stack, int &value) {
    if (IsEmpty(stack)) {
        printf("Stack is empty\n");
        return;
    }
    value = stack.top->key;
}
```

GIẢI PHÓNG BỘ NHỚ CỦA NGĂN XẾP

- Ta liên tục thực hiện tác vụ pop cho đến khi stack rỗng.

```
void FreeStack(STACK &stack) {
     int value;
    while (!IsEmpty(stack)) {
        Pop(stack,value);
    }
}
```

ỨNG DỤNG CỦA NGĂN XẾP

- Duyệt cây và đồ thị
- **Tính toán biểu thức toán học.**
- **Chuyển đổi dạng các biểu thức toán học (infix, postfix, prefix).**
- Quản lý gọi hàm và đệ quy.
- Hỗ trợ Undo/Redo trong các ứng dụng.
- Kiểm tra tính đúng đắn của dấu ngoặc.
- Quản lý bộ nhớ.
- Hỗ trợ chuyển đổi hệ cơ số.
- Backtracking (quay lui).
- ……

Biểu thức trung tố (Infix): 10 / 2 * 4 - 6 + (2 ^ 3 * 4 - 6 * 3 - 4) * 2 + 6

THUẬT TOÁN CHUYỂN INFIX SANG POSTFIX

1.Khởi tạo Stack ***opStack*** rỗng để lưu các toán tử, danh sách ***list*** rỗng để lưu kết quả

2. Tách chuỗi biểu thức infix thành danh sách các ***token***

3. Duyệt các ***token*** từ trái sang phải

3.1. Nếu ***token*** là **toán hạng** thì thêm ***token*** vào cuối ***list***

3.2. Nếu ***token*** là dấu (thì push vào ***opStack***

3.3. Nếu ***token*** là dấu) thì lần lượt pop toán tử trong ***opStack*** và thêm toán tử này vào cuối ***list*** cho đến khi gặp dấu (

Biểu thức hậu tố (Postfix): 10 2 / 4 * 6 - 2 3 ^ 4 * 6 3 * - 4 - 2 * + 6 +

3.4. Nếu token là **toán tử**:

3.4.1. Trong khi đỉnh ***opStack*** là **toán tử** và có **độ ưu tiên ≥ độ ưu tiên** của token thì pop toán tử trong ***opStack*** và **thêm toán tử** này vào cuối ***list***

3.4.2. Push token vào ***opStack***

4. Khi xét hết các ***token*** thì pop tất cả những toán tử còn lại trong ***opStack*** và lần lượt thêm vào cuối ***list***

THUẬT TOÁN CHUYỂN INFIX SANG POSTFIX

- Thứ tự ưu tiên của các toán tử

Toán tử	Thứ tự ưu tiên
∧	3
* , / (:), %	2
+ -	1
( )	0

CHUYỂN BIỂU THỨC TRUNG TỐ SANG HẬU TỐ

- Biểu thức trung tố (Infix): 10 / 2 * 4 - 6 + (2 ^ 3 * 4 - 6 * 3 - 4) * 2 + 6
- Biểu thức hậu tố (Postfix): 10 2 / 4 * 6 - 2 3 ^ 4 * 6 3 * - 4 - 2 * + 6 +

Xem phương pháp chuyển đổi trong giáo trình

Input	Stack	Postfix
10		10
/	/	10
2	/	10 2
*	*	10 2 /
4	*	10 2 / 4
-	-	10 2 / 4 *
6	-	10 2 / 4 * 6
+	+	10 2 / 4 * 6 -
(	+(	10 2 / 4 * 6 -
2	+(	10 2 / 4 * 6 - 2
^	+(^	10 2 / 4 * 6 - 2
3	+(^	10 2 / 4 * 6 - 2 3
*	+(*	10 2 / 4 * 6 - 2 3 ^
4	+(*	10 2 / 4 * 6 - 2 3 ^ 4

Input	Stack	Postfix
-	+(-	10 2 / 4 * 6 - 2 3 ^ 4 *
6	+(-	10 2 / 4 * 6 - 2 3 ^ 4 * 6
*	+(- *	10 2 / 4 * 6 - 2 3 ^ 4 * 6
3	+(- *	10 2 / 4 * 6 - 2 3 ^ 4 * 6 3
-	+(-	10 2 / 4 * 6 - 2 3 ^ 4 * 6 3 * -
4	+(-	10 2 / 4 * 6 - 2 3 ^ 4 * 6 3 * - 4
)	+	10 2 / 4 * 6 - 2 3 ^ 4 * 6 3 * - 4 -
*	+ *	10 2 / 4 * 6 - 2 3 ^ 4 * 6 3 * - 4 -
2	+ *	10 2 / 4 * 6 - 2 3 ^ 4 * 6 3 * - 4 - 2
+	+	10 2 / 4 * 6 - 2 3 ^ 4 * 6 3 * - 4 - 2 * +
6	+	10 2 / 4 * 6 - 2 3 ^ 4 * 6 3 * - 4 - 2 * + 6
	+	10 2 / 4 * 6 - 2 3 ^ 4 * 6 3 * - 4 - 2 * + 6 +

THUẬT TOÁN TÍNH GIÁ TRỊ POSTFIX

Biểu thức hậu tố (Postfix): 10 2 / 4 * 6 - 2 3 ^ 4 * 6 3 * - 4 - 2 * + 6 +

1.Khởi tạo Stack ***s*** rỗng để lưu các toán hạng

2. Tách chuỗi biểu thức postfix thành danh sách các ***token***

3. Duyệt các ***token*** từ trái sang phải

   3.1. Nếu ***token*** là toán hạng thì push ***token*** vào ***s***

   3.2. Nếu ***token*** là toán tử thì:

   3.2.1. b = pop stack s

   3.2.2. a = pop stack s

   3.2.3. kq = a toán tử b

   3.2.4. Push kq vào stack s

4. Khi xét hết các ***token*** thì pop stack s. Đây chính là kết quả biểu thức

TÍNH TOÁN BIỂU THỨC HẬU TỐ

10 2 / 4 * 6 - 2 3 ^ 4 * 6 3 * - 4 - 2 * + 6 +
5 4 * 6 - 2 3 ^ 4 * 6 3 * - 4 - 2 * + 6 +
20 6 - 2 3 ^ 4 * 6 3 * - 4 - 2 * + 6 +
14 2 3 ^ 4 * 6 3 * - 4 - 2 * + 6 +
14 8 4 * 6 3 * - 4 - 2 * + 6 +
14 32 6 3 *- 4 - 2 * + 6 +
14 32 18 - 4 - 2 * + 6 +
14 14 4 - 2 * + 6 +
14 10 2 * + 6 +
14 20 + 6 +
34 6 +
40

Xem phương pháp tính toán biểu thức hậu tố trong giáo trình

BÀI TẬP TẠI LỚP

Bài 1:

- Trình bày từng bước quá trình chuyển đổi biểu thức sau đây từ dạng infix (trung tố) sang dạng postfix (hậu tố) có sử dụng ngăn xếp (stack).

$$(2+3)*5 -4*(10-2^{(4 - 1)})$$

- TÍNH GIÁ TRỊ POSTFIX CỦA BIỂU THỨC TRÊN SAU KHI ĐÃ CHUYỂN ĐỔI CHÚNG TỪ INFIX SANG POSTFIX

Bài 2:

- Trình bày từng bước quá trình chuyển đổi biểu thức sau đây từ dạng infix (trung tố) sang dạng postfix (hậu tố) có sử dụng ngăn xếp (stack).

$$4 * 6 / 2 - (3 - 1+ 4 * 2) – 6 / 3 + 5$$

- Trình bày từng bước quá trình tính giá trị biểu thức postfix vừa chuyển đổi

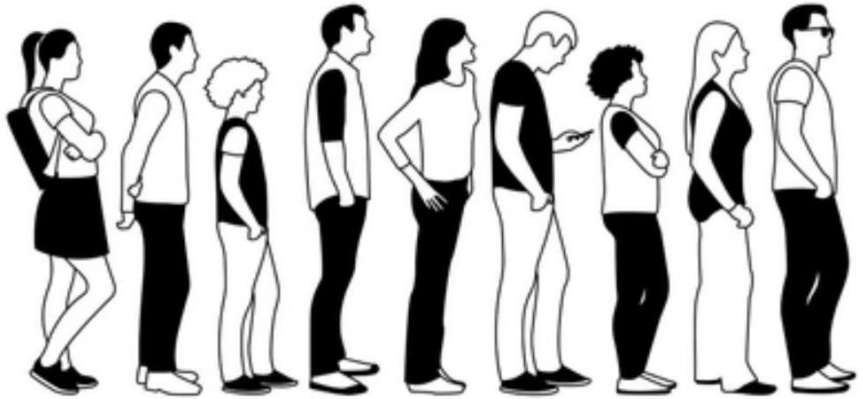
NGĂN XẾP VÀ HÀNG ĐỢI

5.2 - HÀNG ĐỢI (QUEUE)

KHÁI NIỆM HÀNG ĐỢI

KHÁI NIỆM

- Queue (hàng đợi) là một dạng danh sách đặt biệt.
- Thêm các phần tử vào cuối hàng đợi.
- Lấy ra phần tử ở đầu hàng đợi.
- Gọi là cấu trúc FIFO (First In First Out).



- **Hai tác vụ chính của hàng đợi:**

- Enqueue: thêm một phần tử vào cuối của hàng đợi.

- Dequeue: lấy phần tử ở đầu hàng đợi ra khỏi hàng đợi.

KHÁI NIỆM HÀNG ĐỢI

CÁC ỨNG DỤNG CỦA HÀNG ĐỢI

Dùng để giải quyết các vấn đề có cấu trúc FIFO.

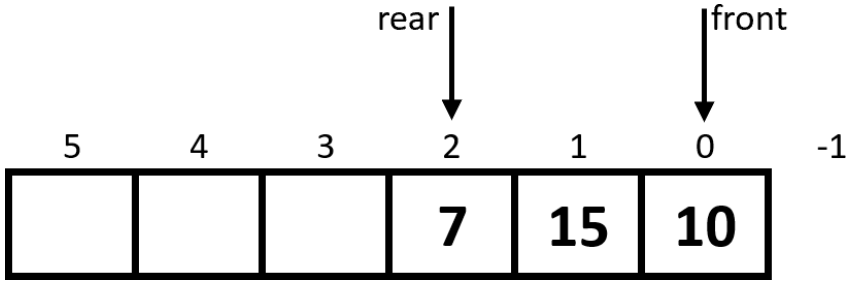
- Xếp hàng tại ngân hàng hoặc siêu thị.
- Hệ thống gọi điện thoại tổng đài.
- In ấn: khi bạn gửi nhiều tài liệu để máy in nó in.
- Hệ thống cấp phát tài nguyên trong máy tính.
- Điều khiển lưu lượng giao thông.
- Quản lý các yêu cầu truy cập máy chủ web.
- Hệ thống tải tập tin từ Internet.
- Ứng dụng trong trò chơi trực tuyến, chờ tới lượt chơi.
- ……

CƠ CHẾ HOẠT ĐỘNG

CƠ CHẾ FIFO

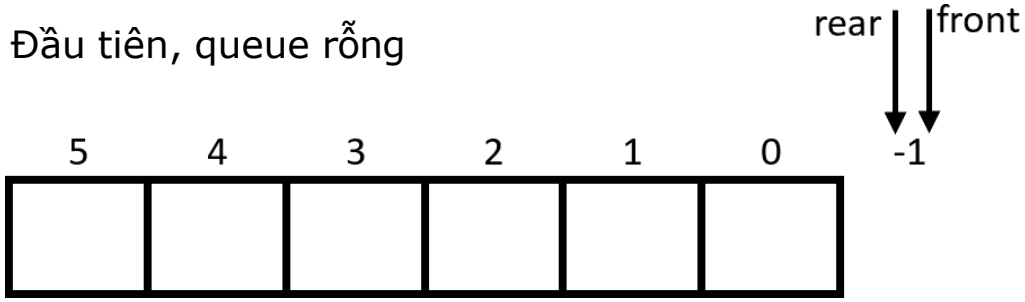
- Hàng đợi lưu trữ các phần tử dữ liệu (item) theo thứ tự.
- Item được thêm tại đầu **rear** của hàng đợi.
- Item được **lấy** ra ở đầu **front** của hàng đợi.

- Ví dụ: một hàng đợi có kích thước tối đa là 6 phần tử.
  - Biến front: nơi phần tử bị lấy ra (dequeue).
  - Biến rear: nơi phần tử mới được thêm vào (enqueue).

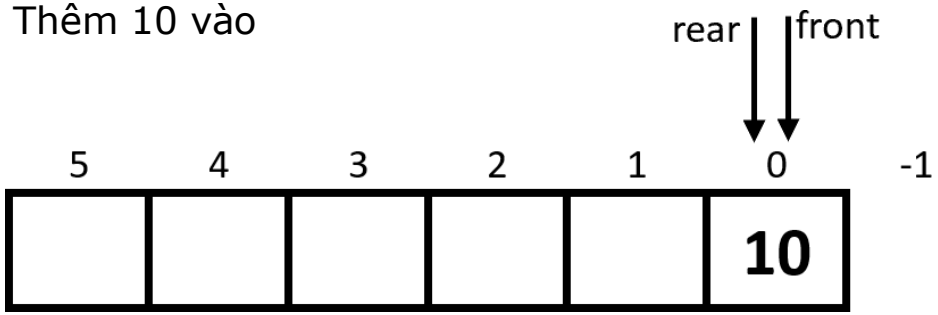


VÍ DỤ MINH HỌA

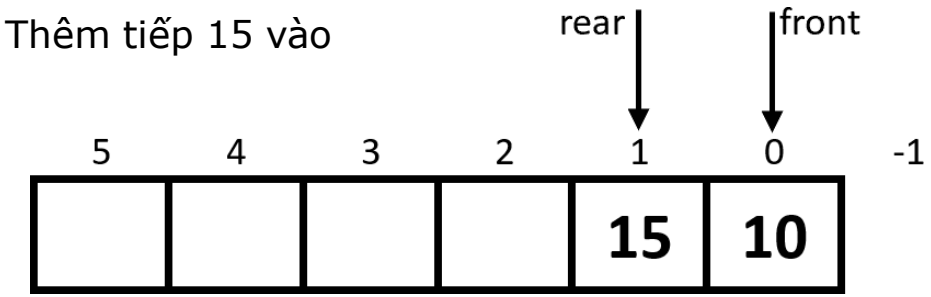
1) Đầu tiên, queue rỗng



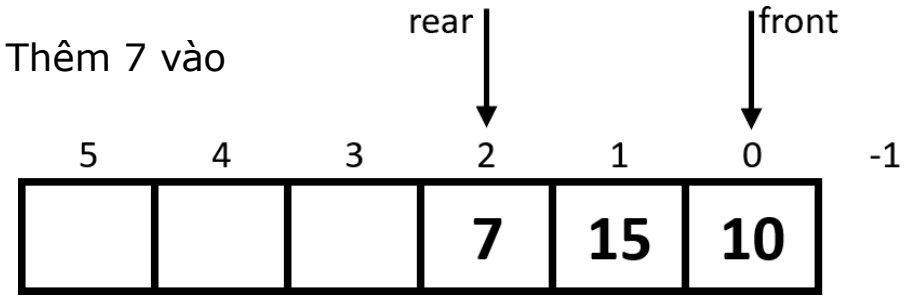
2) Thêm 10 vào



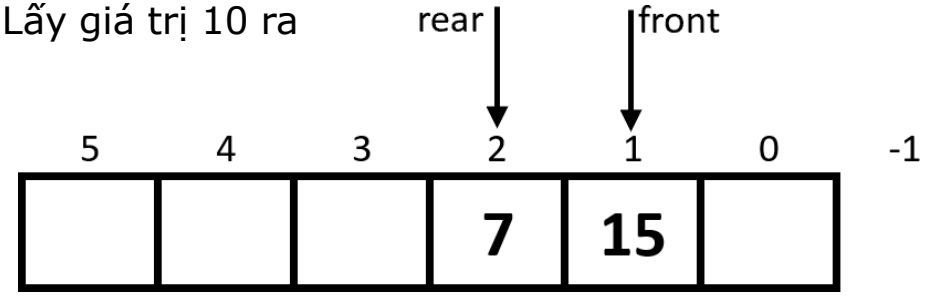
3) Thêm tiếp 15 vào



4) Thêm 7 vào



5) Lấy giá trị 10 ra



CÁC TÁC VỤ CƠ BẢN CỦA HÀNG ĐỢI

- IsEmpty: kiểm tra hàng đợi rỗng hay chưa.
- IsFull: kiểm tra hàng đợi đầy hay chưa.
- EnQueue: thêm một phần tử vào hàng đợi.
- DeQueue: lấy một phần tử ra khỏi hàng đợi.
- Peek: Xem phần tử đầu tiên của hàng đợi mà không loại bỏ nó.

HIỆN TƯỢNG ĐẦY THẬT VÀ ĐẦY GIẢ

- **Hiện tượng đầy thật (True Overflow):**
  - Xảy ra khi hàng đợi **không còn** bất kỳ chỗ trống nào.
  - Thêm vào một phần tử mới sẽ tràn hàng đợi.
- **Hiện tượng đầy giả (False Overflow):**
  - Khi hàng đợi được coi là đầy dù **vẫn còn một số chỗ trống**.
  - Biến **rear** đang ở vị trí cuối cùng.
  - Ta không thể thêm phần tử mới vào.
  - Xảy ra khi hiện thực hàng đợi thẳng với kích thước cố định.

HAI PHƯƠNG PHÁP HIỆN THỰC HÀNG ĐỢI

Có **hai** phương pháp hiện thực hàng đợi (queue):

- **Dùng danh sách liền kề:** hiện thực bằng mảng một chiều thì ta có 2 cách.
  - Phương pháp tịnh tiến.
  - Phương pháp chạy vòng.
- **Dùng danh sách liên kết đơn:**
  - **Thêm** phần tử mới vào sau nút **Tail**.
  - **Lấy** phần tử ra tại nút **Head**.
  - Biến **front** chính là **Head**.
  - Biến **rear** chính là **Tail** của DSLK đơn.

HIỆN THỰC BẰNG DANH SÁCH LIỀN KỀ

XÂY DỰNG CẤU TRÚC DỮ LIỆU

- Mảng một chiều arr: để chứa các phần tử của hàng đợi.
- Biến **front** để quản lý **vị trí đầu** hàng đợi.
- Biến **rear** để quản lý vị trí cuối hàng đợi.
- Biến **size** quản lý số lượng phần tử tối đa.

```
struct queue
{
   int *arr; // mảng dùng để chứa các phần tử của hàng đợi
   int size; // số lượng phần tử tối đa của hàng đợi
   int front; //chỉ số của phần tử tại đầu hàng đợi
   int rear; //chỉ số của phần tử tại cuối hàng đợi
};
typedef struct queue QUEUE; //đặt là QUEUE cho dễ gọi
```

XÂY DỰNG CÁC TÁC VỤ

KHỞI TẠO HÀNG ĐỢI

- Cấp phát bộ nhớ cho mảng động của hàng đợi.
- Khởi tạo các biến **size**, front, **rear**.
- Capacity là số lượng phần tử tối đa của hàng đợi.

```
void Init(QUEUE &queue, int capacity)
{
    queue.arr = (int*)malloc(capacity * sizeof(int));
    queue.size = capacity;
    queue.front = -1;
    queue.rear = -1;
}
```

XÂY DỰNG CÁC TÁC VỤ

KIỂM TRA HÀNG ĐỢI RỖNG

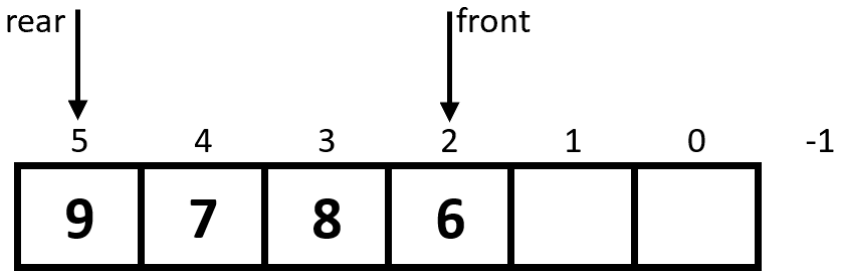
- Hàng đợi rỗng là khi nó không còn chứa bất kỳ một phần tử nào.
- Khi đó, front và rear của nó đều bằng -1.

```
int IsEmpty(QUEUE queue)
{
    if(queue.front == -1)
        return 1; //1 là true, queue đã rỗng
    return 0; //0 là false, queue chưa rỗng
}
```

XÂY DỰNG CÁC TÁC VỤ

ĐẦY GIẢ

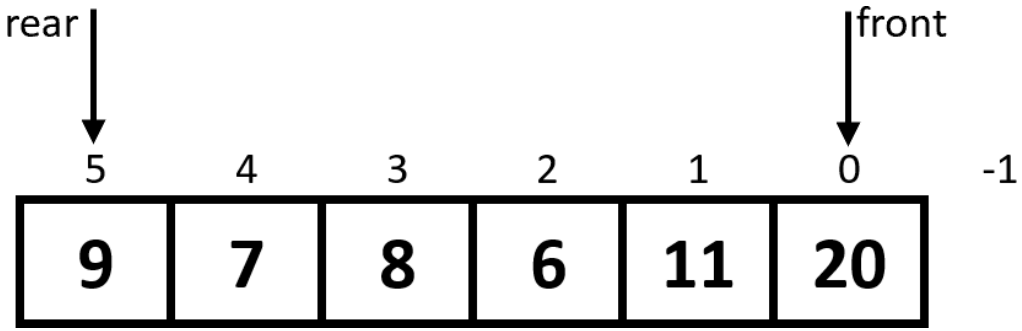
- Rear đã chạm tới vị trí size-1 nhưng phía trước vị trí front vẫn còn ô trống.



Nhận xét: cả hai trường hợp, hàng đợi chỉ thật sự đầy khi số lượng phần tử của nó là size phần tử. **Tức là: (rear-front=size-1).**

ĐẦY THẬT

- **Rear** đã chạm tới vị trí size-1
- **Front** đang đứng tại vị trí 0.
- **Không** còn bất kỳ ô trống nào

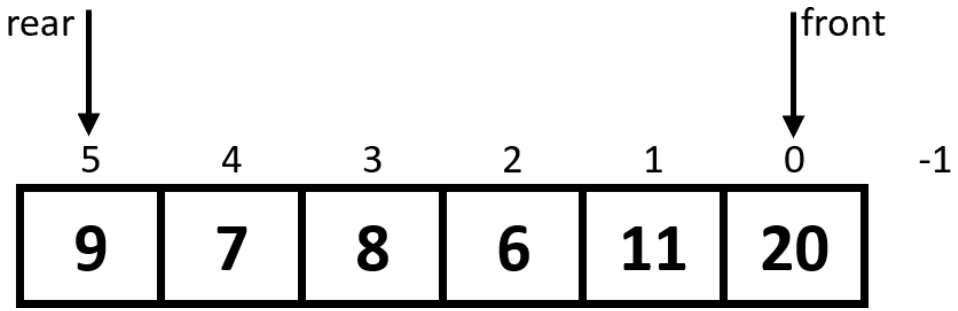


XÂY DỰNG CÁC TÁC VỤ

KIỂM TRA HÀNG ĐỢI ĐẦY

- Tác vụ kiểm tra hàng đợi **đầy** được viết thành hàm như sau.

```
int IsFull(QUEUE queue)
{
    if(queue.rear-queue.front==queue.size-1)
        return 1; //1 là true, queue đã đầy
    return 0; //0 là false, queue chưa đầy
}
```



XÂY DỰNG CÁC TÁC VỤ

THÊM MỘT PHẦN TỬ VÀO HÀNG ĐỢI

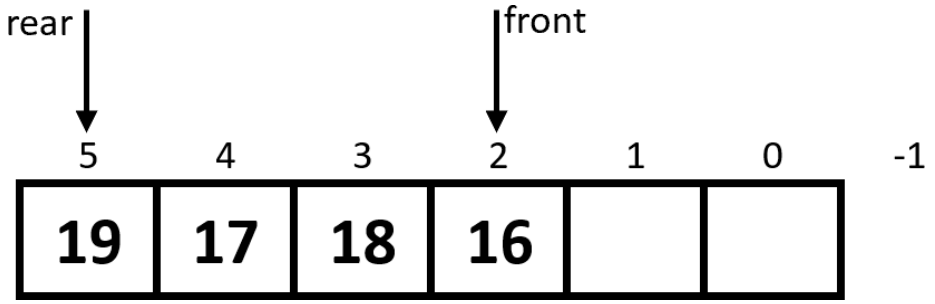
Tác vụ này được gọi là enqueue. Thêm value vào queue.

- **Tăng vị trí** hiện hành của rear lên.
- **Chép value** vào hàng đợi tại vị trí rear mới.

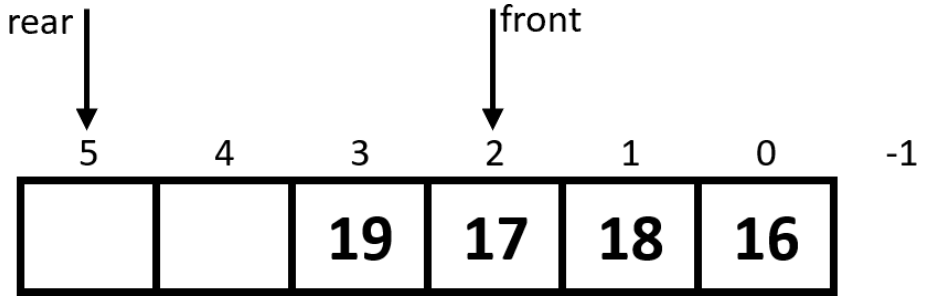
CHÚ Ý:

- Nếu hàng đợi đầy thật thì ta không thể thêm.
- Nếu hàng đợi đầy giả thì ta tịnh tiến tất cả phần tử hướng về phía vị trí 0 rồi mới thêm value vào.

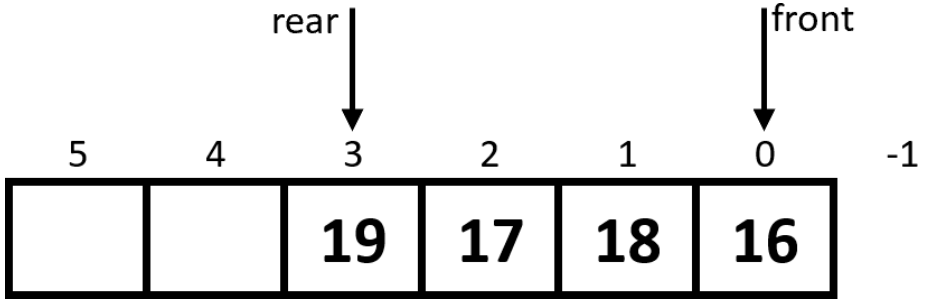
1) Cách thêm một phần tử vào hàng đợi đang đầy giả.



2) **Tịnh tiến** các phần tử ra đầu mảng.



3) Cập nhật lại front và rear.



4) **Thêm** phần tử mới phía **rear**.

XÂY DỰNG CÁC TÁC VỤ

THÊM MỘT PHẦN TỬ VÀO HÀNG ĐỢI

- Tác vụ enqueue được hiện thực thành code như sau:

```
int EnQueue(QUEUE &queue, int new_item)
{
    int i;
    if(IsFull(queue)) return 0;//đầy thật, không thêm được
    if(IsEmpty(queue)) queue.front++;//tăng sẵn front từ 0 lên 1
    if(queue.rear == queue.size-1) //đầy giả
    { //sẽ tịnh tiến các phần tử về phía đầu mảng
        for(i=queue.front;i<queue.size;i++)
            queue.arr[i-queue.front] = queue.arr[i];
            //cập nhật lại rear sau khi tịnh tiến
            queue.rear = queue.size-queue.front-1;
            //cập nhật lại front sau khi tịnh tiến
            queue.front = 0;
    }
    //tăng rear lên một vị trí rồi chép value vào
    queue.arr[++queue.rear]=new_item;
    return 1; //enqueue thành công
}
```

XÂY DỰNG CÁC TÁC VỤ

LẤY MỘT PHẦN TỬ RA KHỎI HÀNG ĐỢI

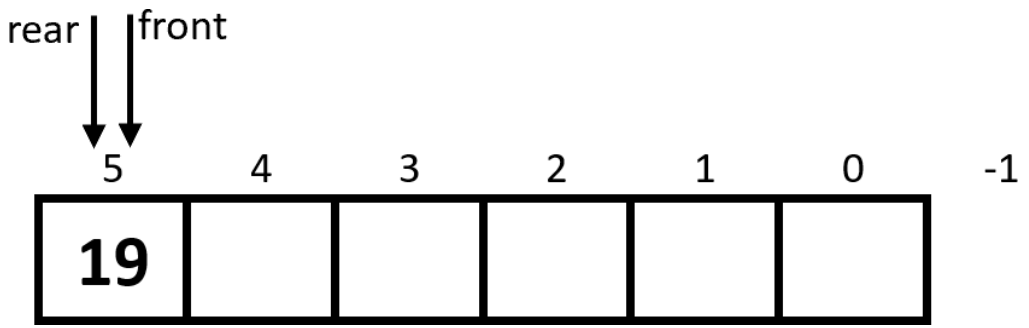
Tác vụ này được gọi là **Dequeue**:
- Lấy phần tử tại **front** gán cho biến value
- **Tăng** biến front lên 1

CHÚ Ý:
- Nếu hàng đợi chỉ còn 1 phần tử mà ta muốn dequeue nó.

Thì có 2 trường hợp như sau.

- Được gọi là tác vụ dequeue.
- Khi hàng đợi chỉ còn **duy nhất một phần tử**, nếu ta lấy phần tử đó ra thì hàng đợi bị rỗng.
- **Phần tử duy nhất** đó có thể nằm ở cuối hàng đợi hoặc ở vị trí lưng lửng bên trong hàng đợi.

Trường hợp 1: phần tử duy nhất nằm tại vị trí **rear=size-1**

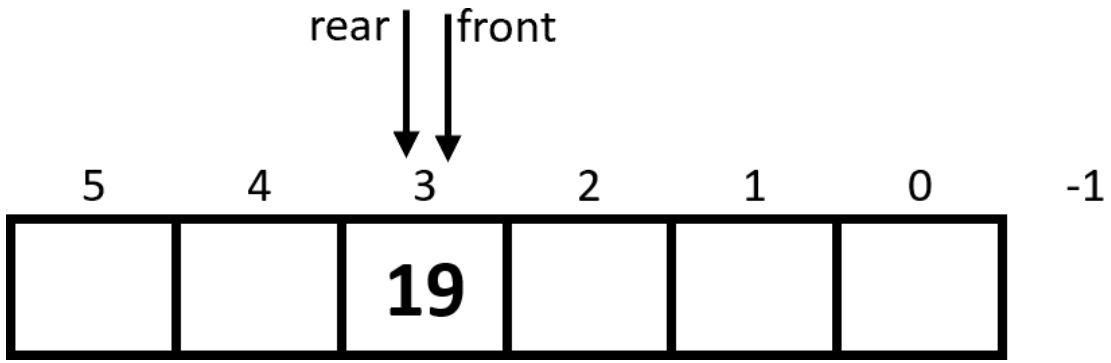


Sau khi dequeue phần tử 19, thì lúc này front=6 và rear=5 (tức là front>rear) ta phải cập nhật lại front và rear đều bằng -1 (vì hàng đợi đã rỗng).

XÂY DỰNG CÁC TÁC VỤ

LẤY MỘT PHẦN TỬ RA KHỎI HÀNG ĐỢI

Trường hợp 2: Phần tử duy nhất nằm vị trí lưng lửng bên trong hàng đợi.



Sau khi lấy ra thì rear=3 và front=4. Tức là front>rear.

Ta phải cập nhật lại front và rear đều bằng -1 (vì hàng đợi đã rỗng).

XÂY DỰNG CÁC TÁC VỤ

LẤY MỘT PHẦN TỬ RA KHỎI HÀNG ĐỢI

- Tác vụ dequeue được hiện thực thành code như sau:

```
int DeQueue(QUEUE &queue, int &value)
{
    if(IsEmpty(queue)) return 0; //dequeue không thành công
    value = queue.arr[queue.front++];//lấy xong thì tăng front
    if(queue.front > queue.rear)//hai trường hợp nói trên
        queue.front = queue.rear = -1;
    return 1;//dequeue thành công
}
```

XÂY DỰNG CÁC TÁC VỤ

TRUY CẬP PHẦN TỬ Ở ĐẦU HÀNG ĐỢI

- Được gọi là tác vụ peek. Peek chỉ đọc giá trị tại vị trí front mà không xóa nó ra khỏi hàng đợi.

```
int Peek(QUEUE queue,int &value) {
    if (IsEmpty(queue)) return 0; //peek không thành công
    value = queue.arr[queue.front];
    return 1; //peek thành công
}
```

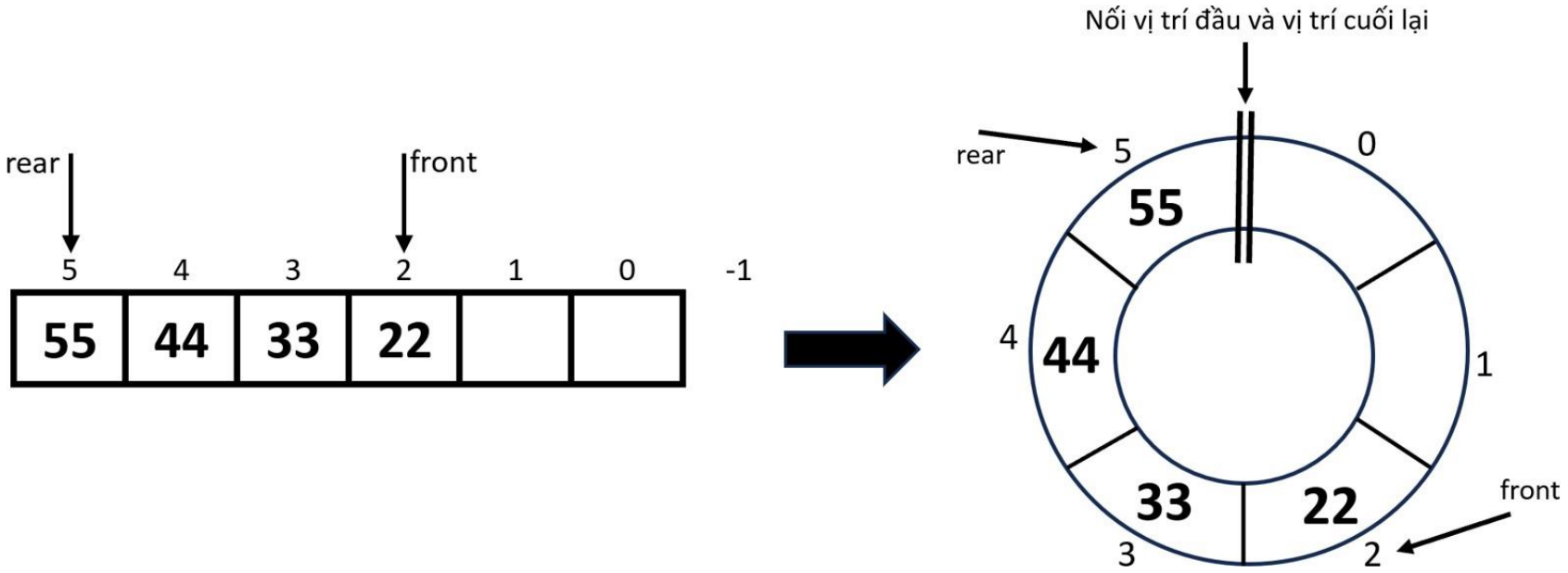
GIẢI PHÓNG BỘ NHỚ CỦA HÀNG ĐỢI

- Nếu không dùng hàng đợi nữa thì ta phải giải phóng bộ nhớ mà nó đang chiếm dụng

```
void FreeQueue(QUEUE &stack) {
    free(queue.arr);
}
```

HÀNG ĐỢI CHẠY VÒNG

- Hàng đợi được hiện thực nói trên được gọi là hàng đợi thẳng (Linear Queue).
- Khi Linear Queue bị đầy giả thì phải tiến hành động tác tịnh tiến tốn kém.
- Để cải tiến, ta hãy hiện thực hàng đợi bằng phương pháp chạy vòng (Circular Queue).

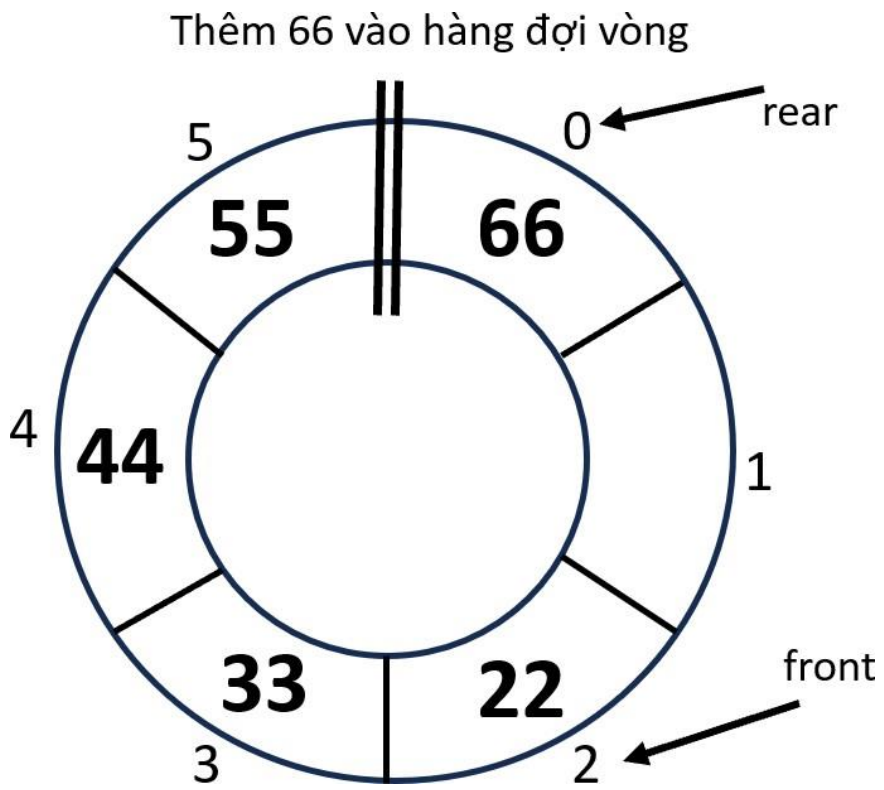


HÀNG ĐỢI CHẠY VÒNG

- Khi hàng đợi chạy vòng bị đầy giả.
- Ta muốn thêm 66 vào.

- Cho rear quay về vị trí mới là 0
- Thêm 66 vào vị trí mới đó.

- Không cần tịnh tiến hàng đợi như cách cũ.
- Đầy giả không tồn tại trong hàng đợi chạy vòng.



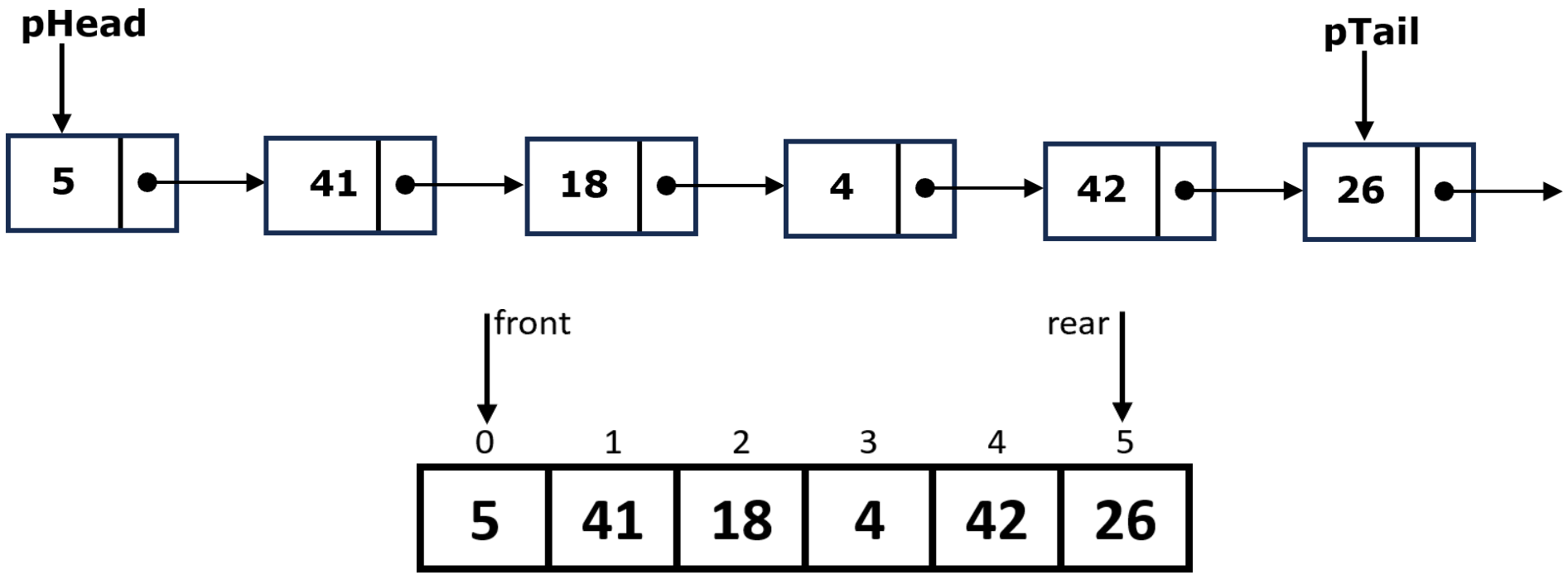
Hàng đợi chạy vòng cũng có các tác vụ cơ bản như hàng đợi thẳng (Linear Queue).

Việc viết code để hiện thực các tác vụ cho hàng đợi chạy vòng được xem như bài tập thực hành.

HIỆN THỰC BẰNG DANH SÁCH LIÊN KẾT ĐƠN

Sự tương quan giữa danh sách liên kết đơn và hàng đợi:

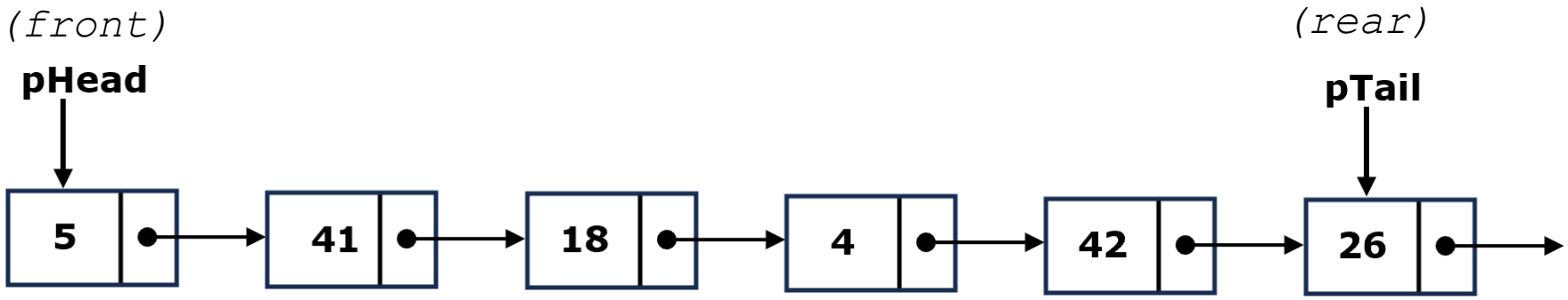
- Biến front tương đương với pHead.
- Biến rear tương đương với pTail.



HIỆN THỰC BẰNG DANH SÁCH LIÊN KẾT ĐƠN

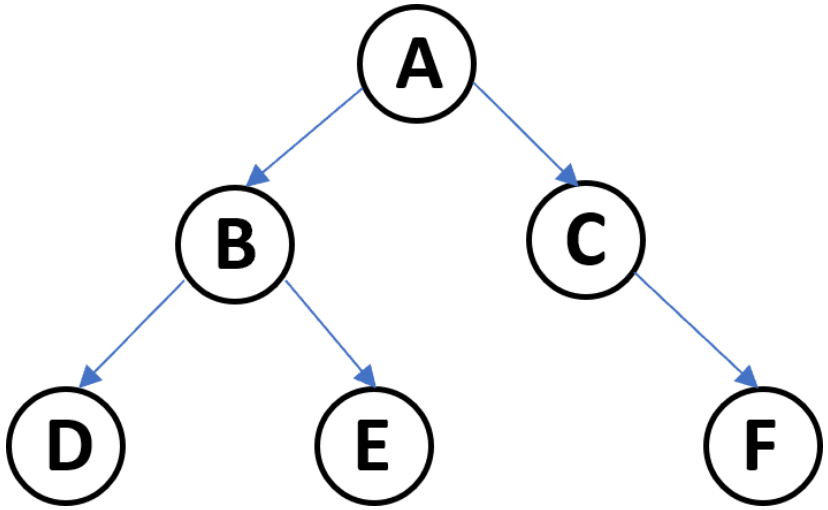
ĐẶC ĐIỂM CỦA HÀNG ĐỢI HIỆN THỰC BẰNG DSLK ĐƠN

- Enqueue: thêm vào **cuối** danh sách liên kết.
- Dequeue: phần tử ở **đầu** danh sách liên kết được loại bỏ.
- Dynamic Size: hàng đợi có thể mở rộng hoạc thu hẹp linh hoạt.
- Không lãng phí bộ nhớ: không gian bộ nhớ chỉ được cấp phát khi cần thiết.
- Không bị hiện tượng đầy giả.



MỘT BÀI TOÁN ỨNG DỤNG CỦA HÀNG ĐỢI

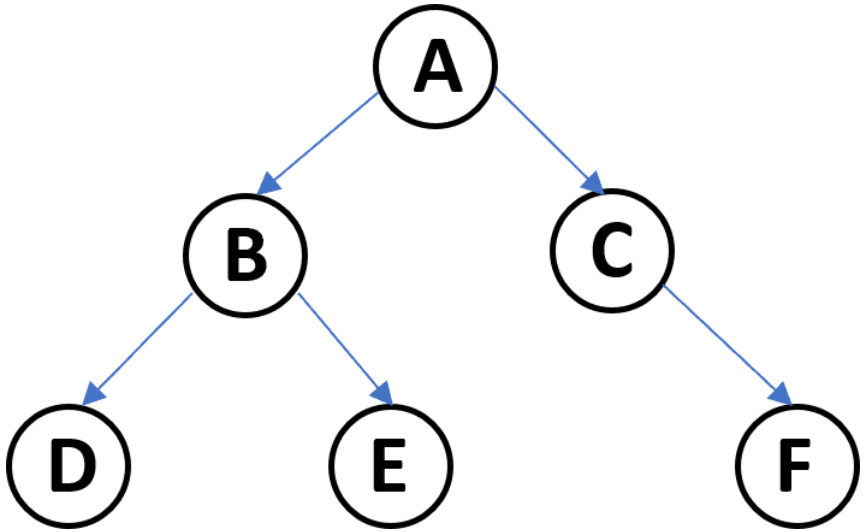
- Dùng hàng đợi để chạy thuật toán tìm kiếm theo chiều rộng (Breadth-First Search, BFS) trên một đồ thị dạng cây.
- Đây là cách tìm kiếm các đỉnh của đồ thị bằng cách duyệt qua các đỉnh lân cận trước khi chuyển sang các đỉnh ở mức tiếp theo.
- Bài toán cụ thể, cho một đồ thị đơn giản như sau:



MỘT BÀI TOÁN ỨNG DỤNG CỦA HÀNG ĐỢI

SƠ LƯỢC THUẬT TOÁN BFS

- Bắt đầu từ đỉnh **gốc A**, đánh dấu **nó là đã thăm** và đưa vào hàng đợi.
- Lặp lại các bước sau cho đến khi hàng đợi **trống**:
  - **Lấy một đỉnh** khỏi hàng đợi (gọi nó là đỉnh hiện tại).
  - Đối với mỗi đỉnh **kề** của **đỉnh hiện tại**, nếu đỉnh đó **chưa** được thăm, đánh dấu nó là **đã thăm** và đưa nó vào hàng đợi.



Khởi tạo: Đỉnh gốc A được thêm vào hàng đợi.

- Hàng đợi: [A]
- Đã thăm: {A}

CHẠY TỪNG BƯỚC BFS

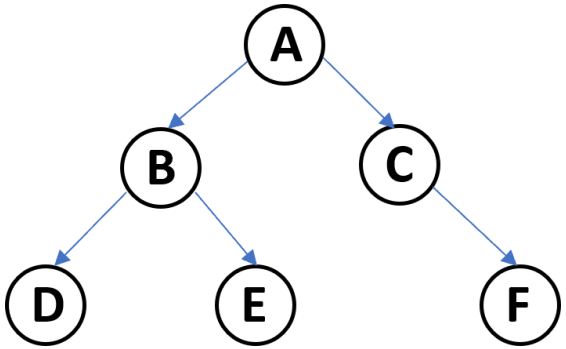
Lặp lại các bước cho đến khi hàng đợi trống:

1) Lấy A khỏi hàng đợi, thăm các đỉnh kề B và C, đánh dấu chúng là đã thăm và thêm vào hàng đợi.

- Hàng đợi: [B, C]
- Đã thăm: {A, B, C}

2) Lấy B khỏi hàng đợi, thăm các đỉnh kề D và E, đánh dấu chúng là đã thăm và thêm vào hàng đợi.

- Hàng đợi: [C, D, E]
- Đã thăm: {A, B, C, D, E}



3) Lấy C khỏi hàng đợi, thăm đỉnh kề F, đánh dấu nó là đã thăm và thêm vào hàng đợi.

- Hàng đợi: [D, E, F]
- Đã thăm: {A, B, C, D, E, F}

4) Lấy D khỏi hàng đợi, không có đỉnh kề nào chưa được thăm.

- Hàng đợi: [E, F]
- Đã thăm: {A, B, C, D, E, F}

5) Lấy E khỏi hàng đợi, không có đỉnh kề nào chưa được thăm.

- Hàng đợi: [F]
- Đã thăm: {A, B, C, D, E, F}

6) Lấy F khỏi hàng đợi, không có đỉnh kề nào chưa được thăm.

- Hàng đợi: [] (trống)
- Đã thăm: {A, B, C, D, E, F}

Thứ tự các đỉnh được thăm: **A, B, C, D, E, F**

BÀI TẬP

BÀI 1: Trình bày từng bước quá trình chuyển đổi biểu thức sau đây từ dạng infix (trung tố) sang dạng postfix (hậu tố) có sử dụng ngăn xếp (stack).

4 * 6 / 2 - (3 - 1+ 4 * 2) – 6 / 3 + 5

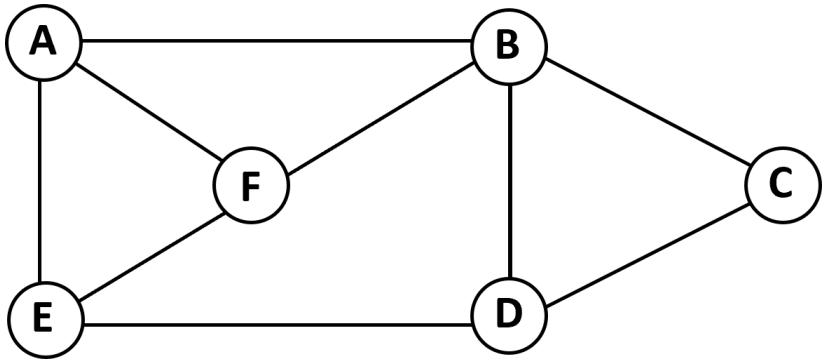
BÀI 2: Trình bày từng bước quá trình tính giá trị biểu thức postfix vừa chuyển đổi.

BÀI 3: Hãy viết code chuyển đổi một số nguyên từ hệ thập phân sang hệ nhị phân có sử dụng ngăn xếp. Sau đó viết code chuyển đổi một số nguyên từ hệ thập phân sang hệ bất kỳ (gồm có hệ nhị phân, hệ bát phân và hệ thập lục phân).

BÀI 4: Hãy viết code tính tổng $S=1+2+3+\ldots+n$ bằng phương pháp đệ quy. Sau đó áp dụng ngăn xếp, viết code tính lại tổng S bằng phương pháp khử đệ quy có sử dụng ngăn xếp.

BÀI TẬP

BÀI 5: Xem lại ví dụ về duyệt đồ thị bằng giải thuật BFS, duyệt đồ thị theo chiều rộng (có sử dụng hàng đợi) đã nói trong bài học. Hãy áp dụng giải thuật BFS đó để duyệt đồ thị sau đây. Bắt đầu duyệt từ đỉnh A.



BÀI 6: Hãy viết code hiện thực hàng đợi theo phương pháp chạy vòng.

BÀI 7: Hãy viết code hiện thực hàng đợi sử dụng danh sách liên kết đơn.

BÀI 8: Hãy sử dụng ngăn xếp (stack) chạy từng bước bài toán Tháp Hà Nội (không được sử dụng đệ quy).

Q&A!